MCU System Clock & RCC

Embedded Controller

Prof. Young-Keun Kim

MCU System Clock

- **Contents**
  - System clock
  - External and Internal Clock
  - PLL
  - Real Time Clock
  - Clock for Peripheral
  - Reset & Clock Control (RCC) Register

MCU System Clock

- **System Clocks (SYSCLK)**
  - 3 different clock sources to drive system clocks (HSx: High-Speed x)
    - ✓ HSI oscillator clock : RC based, Internal Oscillator in CPU (default)
    - ✓ HSE oscillator clock : External Crystal (OFF at reset)
    - ✓ PLL clock

- **Secondary clock sources**
  - LSI(Low-Speed Internal): RC based, internal MCU, 32kHz
  - LSE(Low-Speed External): 32.768kHz low-speed external crystal to drive Real-Time-Clock

MCU System Clock

- **STM32F411- Nucleo-64**

  Core up to 100MHz (PLL)

  LSE: External Crystal 32kHz

  HSI= 16MHz

  HSE= 8MHz

- **STM32F401- Nucleo-64**

  Core up to 84MHz (PLL)

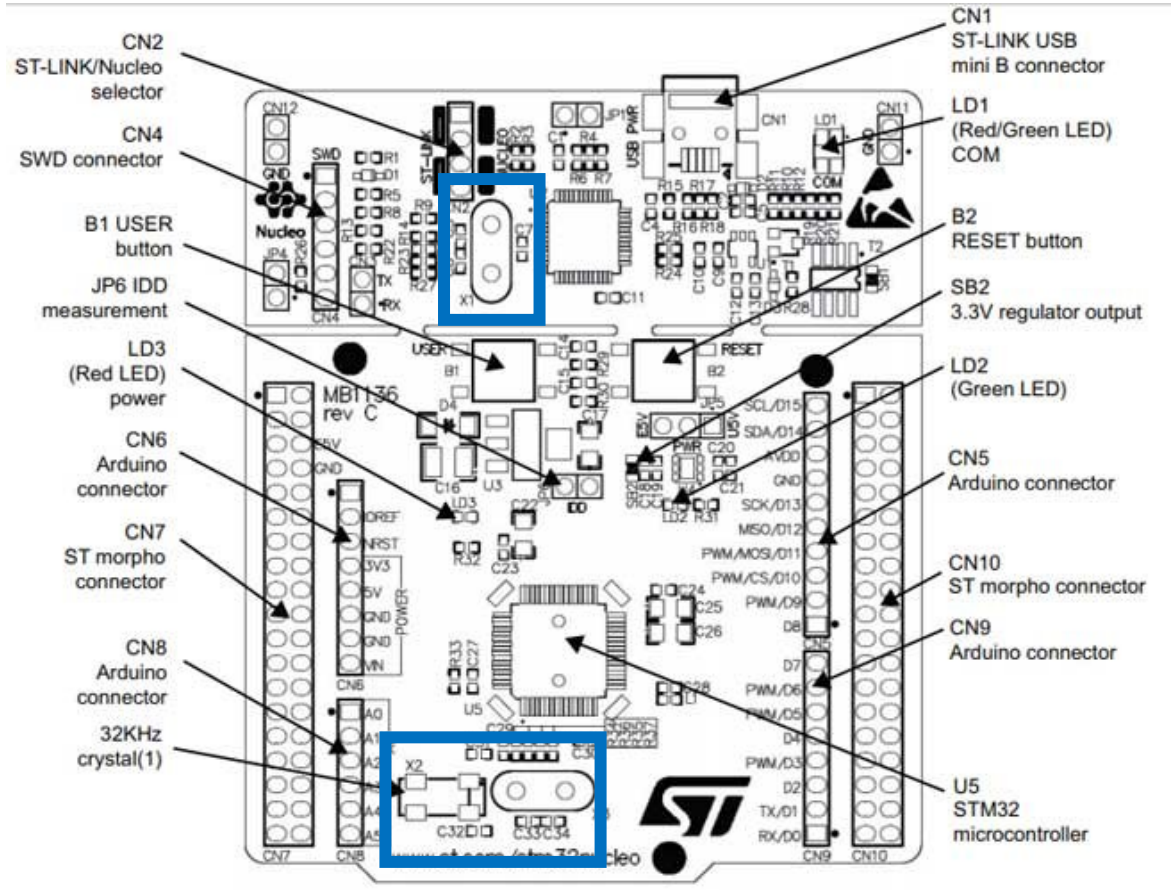
  LSE: External Crystal 32kHz

  HSI= 16MHz

  HSE= 8MHz

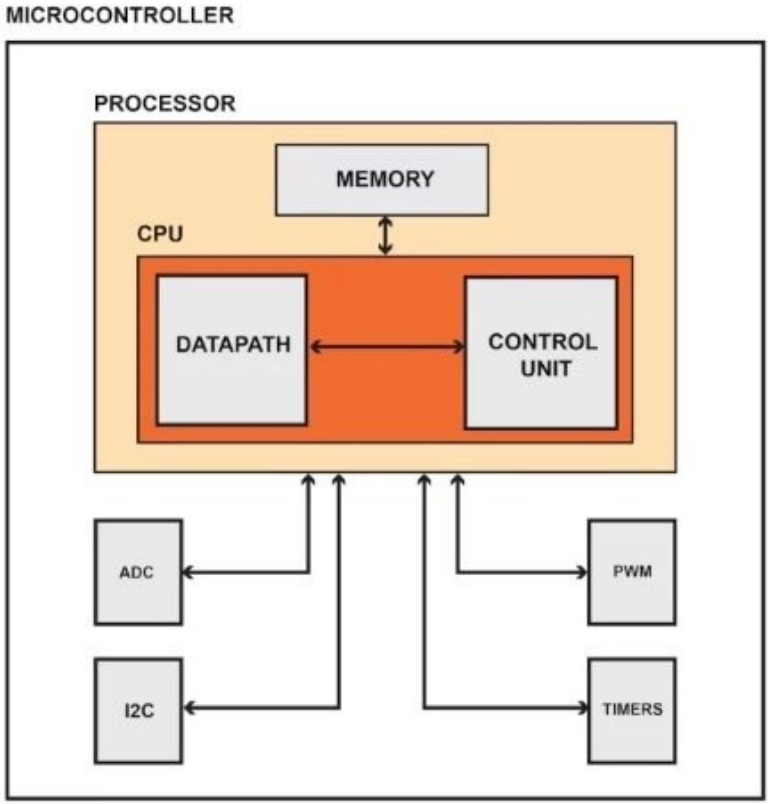
- **Arduino Uno/Mega**

  Core up to 16MHz (PLL)



MCU System Clock

- **Peripherals may need different CLK frequency**
  - All peripheral use the same system CLK source (HSI, HSE, PLL)
  - But, we can set different CLK frequency for each peripherals
    - PWM, GPIO, ADC : can have different CLK period
  - System CLK is controlled in RCC register

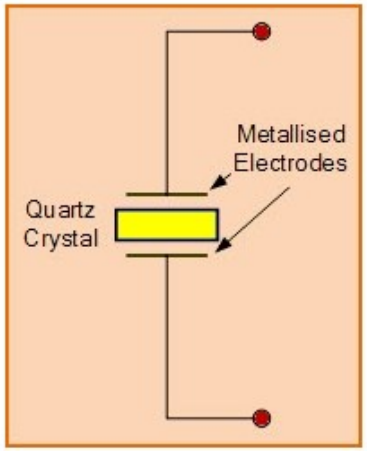


HSE

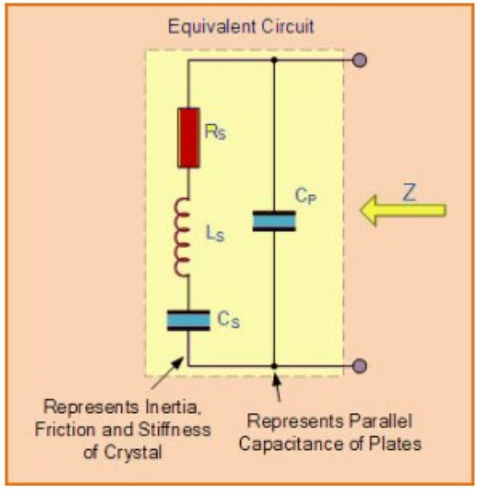
- **HSE (high-speed external)**
- Highly accurate clock
- External Crystal : stm32f411 uses 8MHz
  - 8MHz crystal oscillator is commonly used in MCU
- Crystal Oscillator: Quartz crystal
- Takes longer time for start-up HSI
- Based on the principle of the inverse piezoelectric effect (current→oscillation)



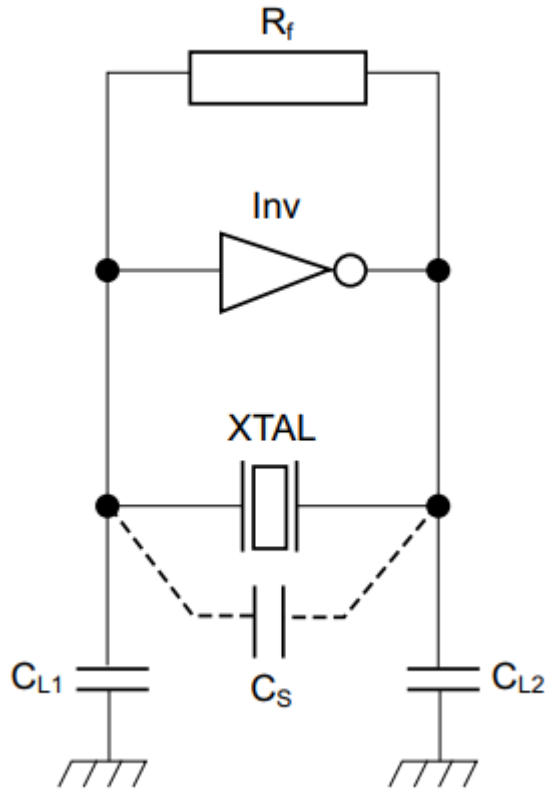
Quartz Crystal Oscillator



Electronic Symbol for Piezoelectric Crystal Resonator

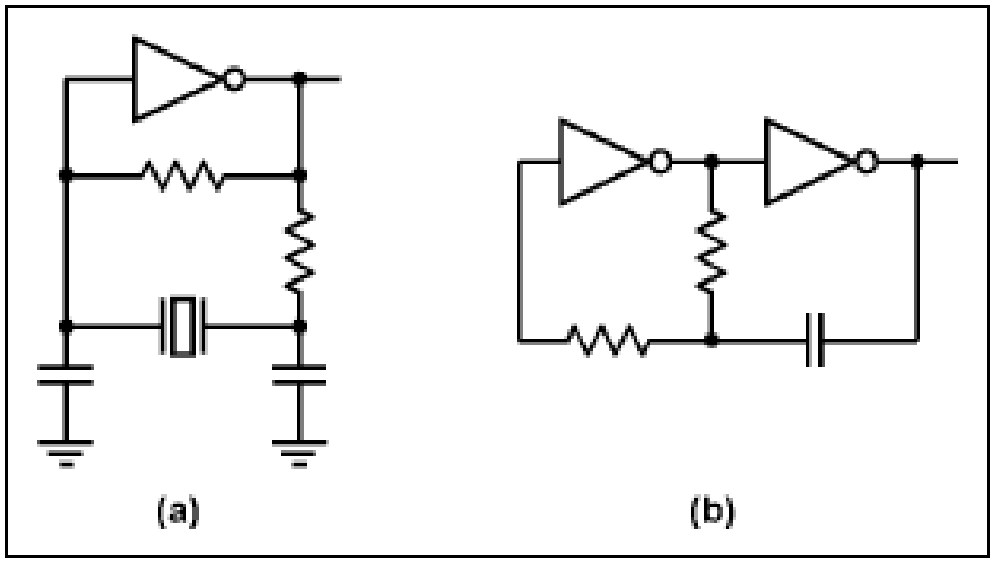


Equivalent Circuit Diagram of Quartz Crystal



HSI

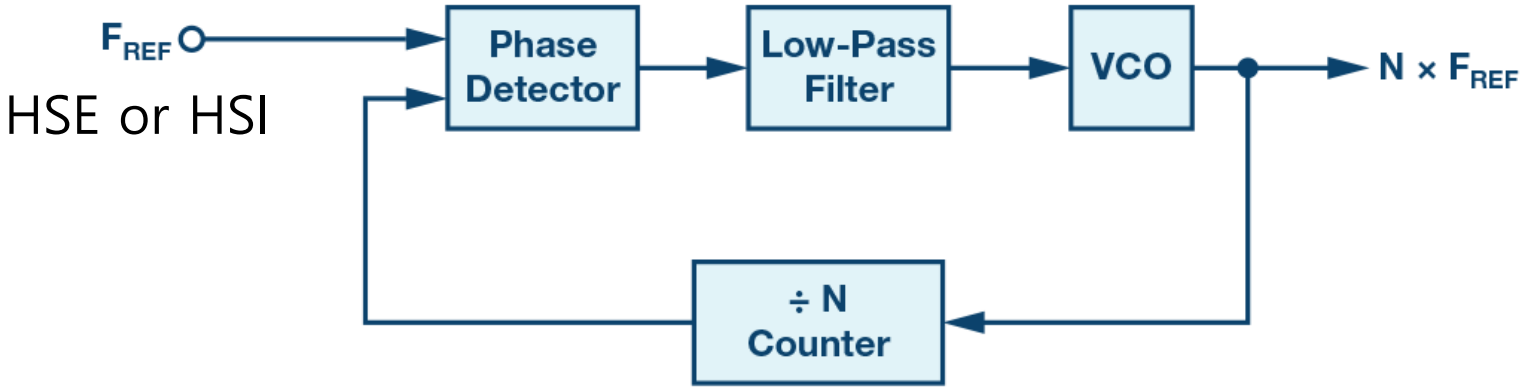
- **HSI(High-Speed Internal)**
  - Generated from an internal 16 MHz RC oscillator, internal MCU
  - Can be used directly as a system clock or used as PLL input
  - Faster/Cheaper but Less Accurate than HSE
  - Must be stable: at startup, need to wait for stable CLK



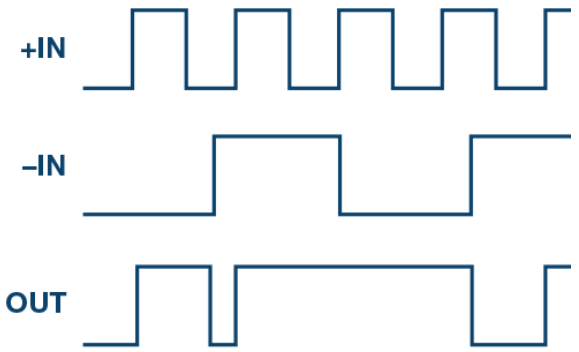
Crystal Oscillator **RC Oscillator**

PLL

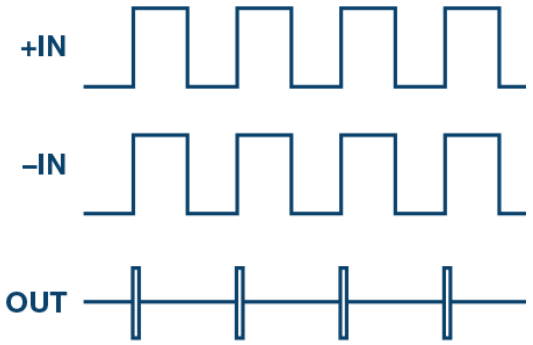
- **PLL (Phase-locked Loop)**
  - Closed-loop frequency-control system based on the phase difference between the input clock signal and the feedback clock signal of a controlled oscillator (VCO)
  - Phase frequency detector (PFD)
    - Comparator that gives a voltage proportional to differences in two frequencies
  - Loop-Pass Filter
    - Eliminate high-frequency component and noise
  - Voltage-controlled-oscillator (VCO)
    - Oscillated signals with a multiple of ref_freq
    - Adjust the phase



[analog.com]



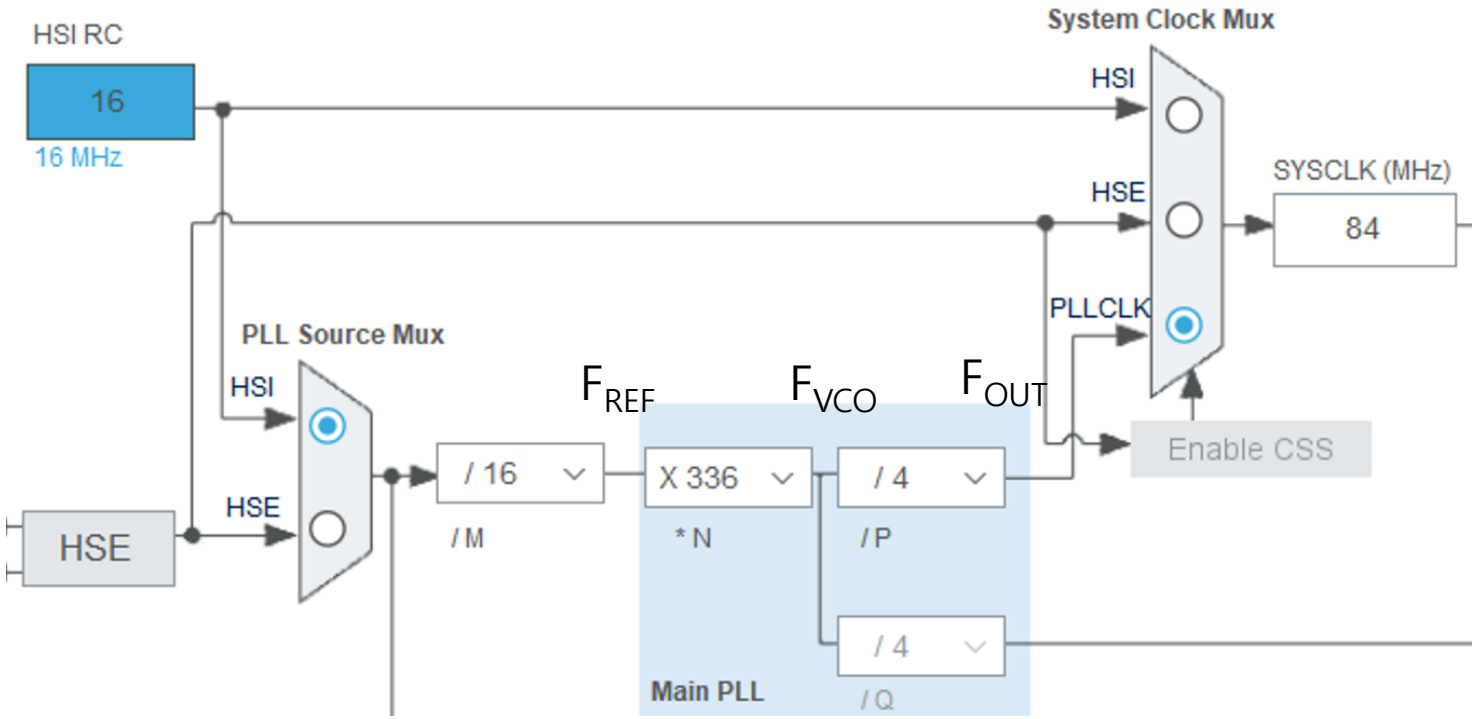
Out of phase



In phase

PLL

- **System Clock: PLL**
  - Counters: feedback counter (M), pre-scale counter (N), and post-scale counters(P)
    - $f_{(VCO\ clock)} = f_{(PLL\ clock\ input)} \times (PLLN / PLLM)$
    - $f_{(PLL\ general\ clock\ output)} = f_{(VCO\ clock)} / PLLP$



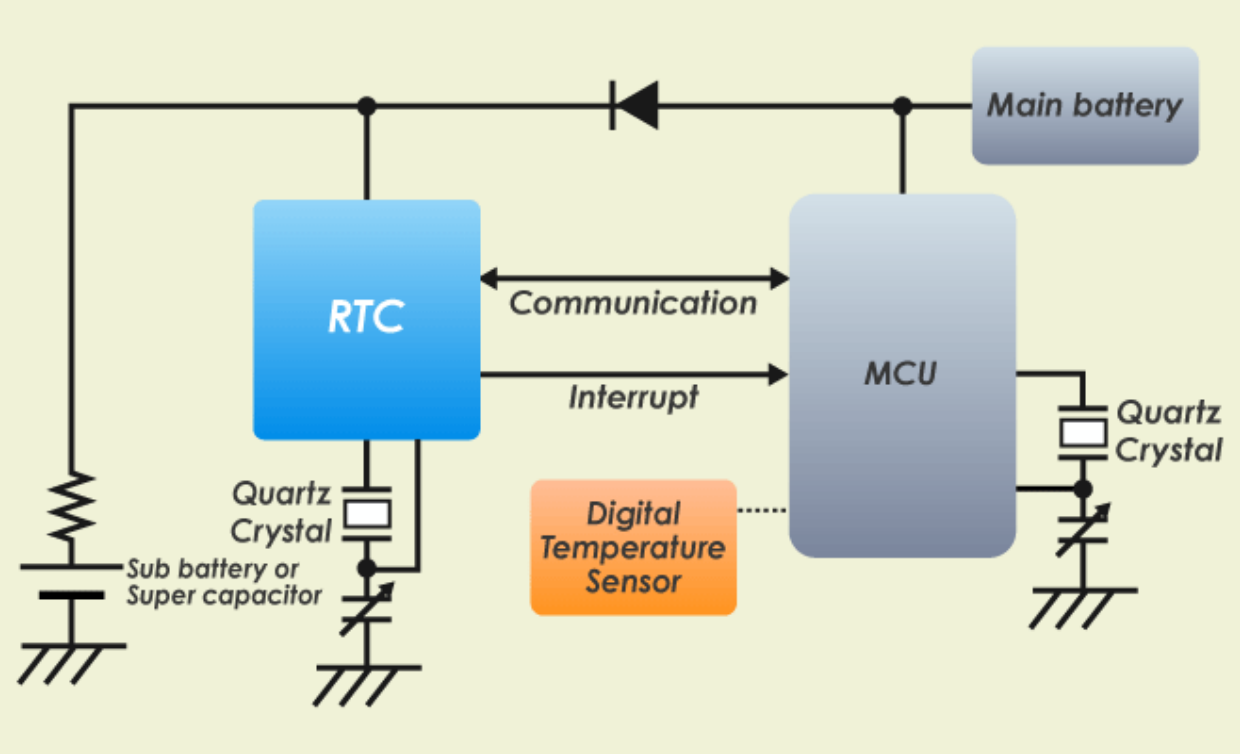
$F_{REF} = F_{IN} / M$
$F_{VCO} = F_{REF} \times N = F_{IN}(N/M)$
$F_{OUT} = F_{VCO}/P$

F_{REF} = Reference
F_{VCO} = VCO frequency
F_{OUT} = Output frequency

N: Multiplication counter
M: Divider counter
C: Post Scaler

RTC

- **Real Time Clock**
  - Real-time clock is a clock function used by systems that measure time even when the main device (such as MCU) is powered off.
    - Date/Time etc
  - 32.768kHz low-speed external crystal to drive RTC clock



System Clock:
Reset and Clock Control (RCC)
Register Description

Embedded Controller

Prof. Young-Keun Kim

RCC : How to Setup for GPIO Clock (AHB)

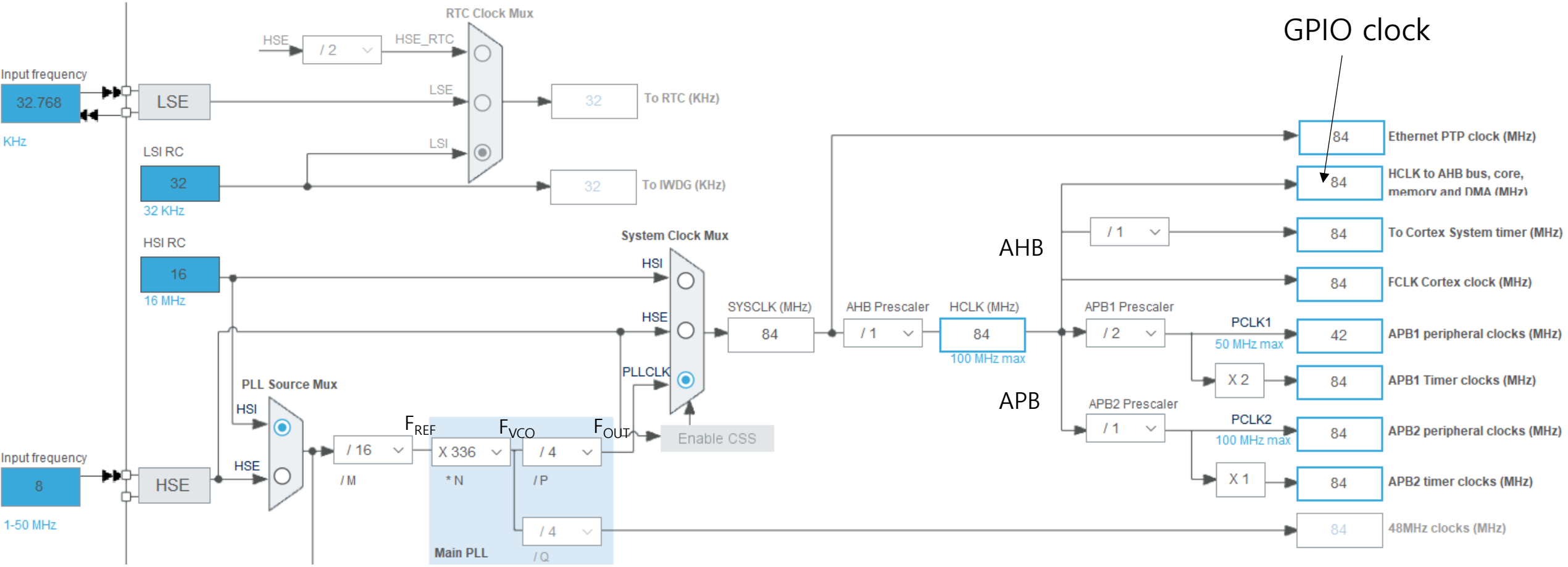
- **AHB (High-Speed Bus)**
  - **AHB**
    - GPIOA~H
    - DMA
    - USB

- **APB (Peripheral Bus)**
  - **APB(High speed <100MHz)**
    - TIMER
  - **APB(Low speed <50MHz)**
    - USART
    - I2C
    - SPI

RCC: Clock Control Register

- **Configuring PLL in CubeMx**

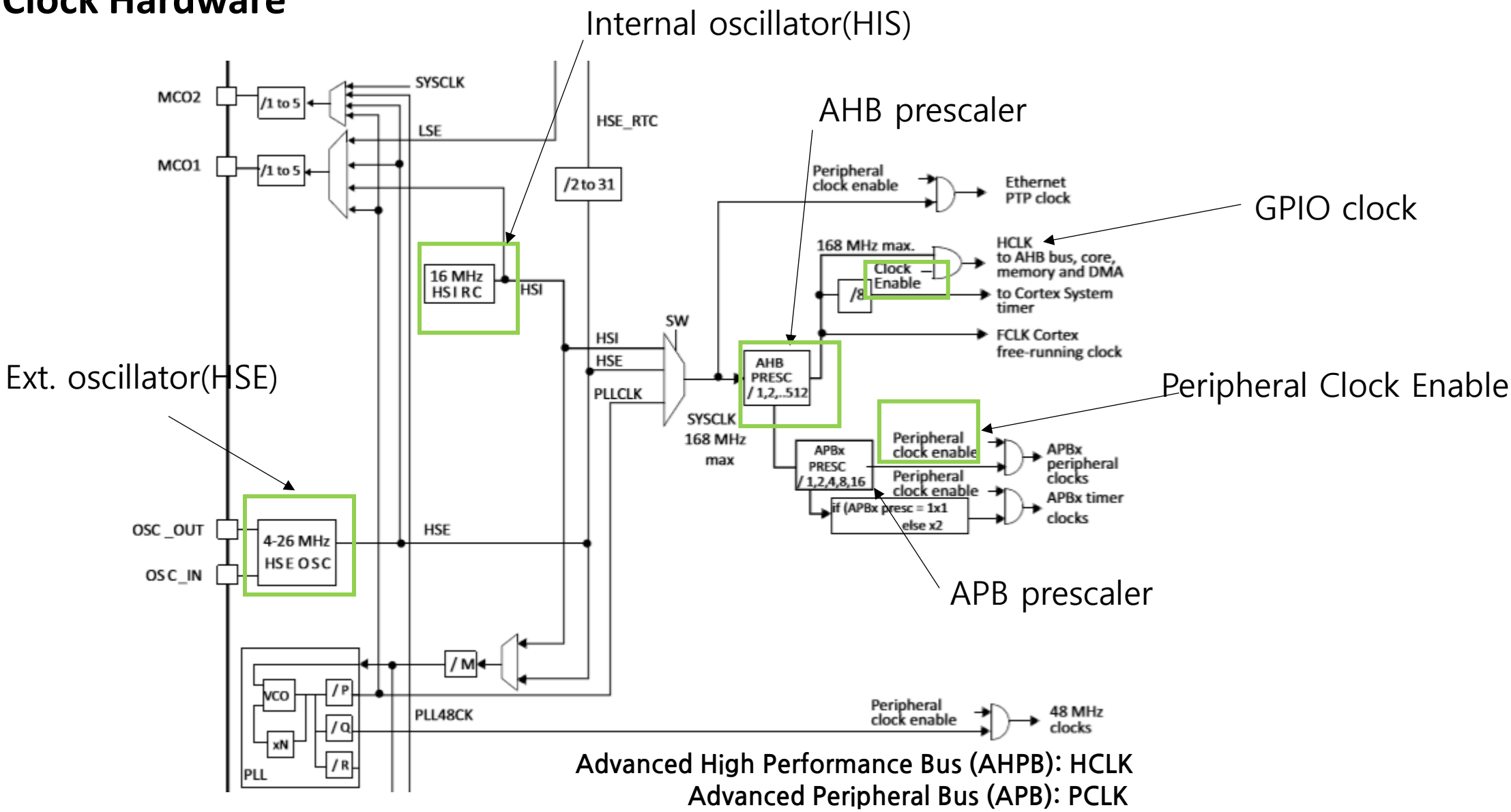
GPIO peripheral is AHB



RCC: Clock Control Register

<RM, Ch 6. Reset and clock control (RCC), pg. 94>

- **System Clock Hardware**

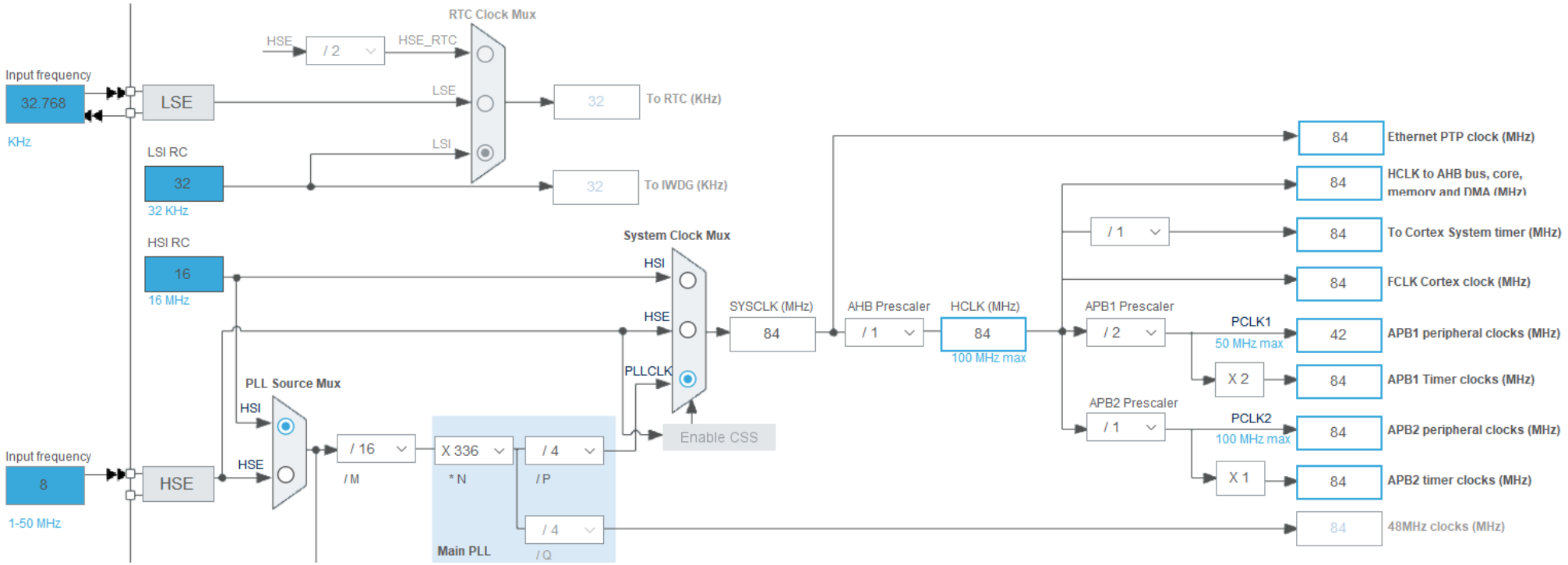


RCC : How to Setup for GPIO Clock (AHB)

- **How to Setup and Enable for GPIO Clock**
  - **Option 1. HSI, Internal Clock (slower)**
    - Enable and Select HSI
    - Configure AHB Prescaler
    - Enable GPIOx Clock (RCC_AHB1ENR)

Option 2. PLL+HSE(HSI) (faster)

- PLL Source = HSE(HIS)
- Enable and Select PLL
- Configure AHB Prescaler
- Enable GPIOx Clock (RCC_AHB1ENR)



RCC Register

RCC Register Memory Address

<RM, Ch 6. Reset and clock control (RCC), pg. 94>

6.3 RCC registers 161
6.3.1 RCC clock control register (RCC_CR) 161
6.3.2 RCC PLL configuration register (RCC_PLLCFGR) 163
6.3.3 RCC clock configuration register (RCC_CFGR) 165
6.3.4 RCC clock interrupt register (RCC_CIR) 167
6.3.5 RCC AHB1 peripheral reset register (RCC_AHB1RSTR) 170
6.3.6 RCC AHB2 peripheral reset register (RCC_AHB2RSTR) 173
6.3.7 RCC AHB3 peripheral reset register (RCC_AHB3RSTR) 174
6.3.8 RCC APB1 peripheral reset register (RCC_APB1RSTR) 174
6.3.9 RCC APB2 peripheral reset register (RCC_APB2RSTR) 178
6.3.10 RCC AHB1 peripheral clock register (RCC_AHB1ENR) 180
6.3.11 RCC AHB2 peripheral clock enable register (RCC_AHB2ENR) 182
6.3.12 RCC AHB3 peripheral clock enable register (RCC_AHB3ENR) 183
6.3.13 RCC APB1 peripheral clock enable register (RCC_APB1ENR) 183
6.3.14 RCC APB2 peripheral clock enable register (RCC_APB2ENR) 187
6.3.15 RCC AHB1 peripheral clock enable in low power mode register (RCC_AHB1LPENR) 189

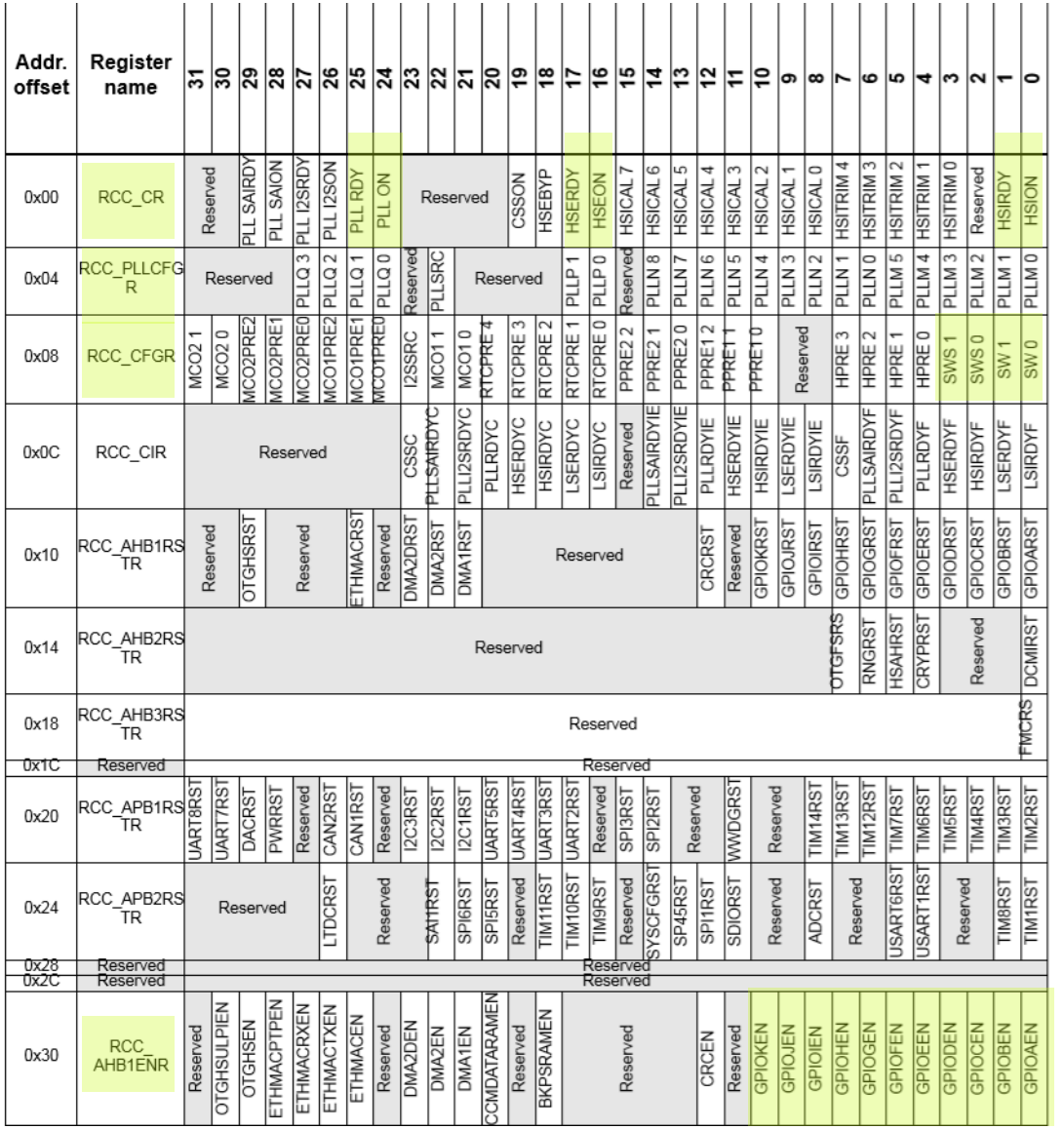
<stm32f411xe.h>

```
/**
  * @brief Reset and Clock Control
  */

typedef struct
{
  __IO uint32_t CR;            /*!< RCC clock control register,
  __IO uint32_t PLLCFGR;       /*!< RCC PLL configuration register,
  __IO uint32_t CFGR;          /*!< RCC clock configuration register,
  __IO uint32_t CIR;           /*!< RCC clock interrupt register,
  __IO uint32_t AHB1RSTR;      /*!< RCC AHB1 peripheral reset register,
  __IO uint32_t AHB2RSTR;      /*!< RCC AHB2 peripheral reset register,
  __IO uint32_t AHB3RSTR;      /*!< RCC AHB3 peripheral reset register,
  uint32_t      RESERVED0;     /*!< Reserved, 0x1C
  __IO uint32_t APB1RSTR;      /*!< RCC APB1 peripheral reset register,
  __IO uint32_t APB2RSTR;      /*!< RCC APB2 peripheral reset register,
  uint32_t      RESERVED1[2];  /*!< Reserved, 0x28-0x2C
  __IO uint32_t AHB1ENR;       /*!< RCC AHB1 peripheral clock register,
  __IO uint32_t AHB2ENR;       /*!< RCC AHB2 peripheral clock register,
  __IO uint32_t AHB3ENR;       /*!< RCC AHB3 peripheral clock register,
  uint32_t      RESERVED2;     /*!< Reserved, 0x3C
  __IO uint32_t APB1ENR;       /*!< RCC APB1 peripheral clock enable register,
  __IO uint32_t APB2ENR;       /*!< RCC APB2 peripheral clock enable register,
  uint32_t      RESERVED3[2];  /*!< Reserved, 0x48-0x4C
  __IO uint32_t AHB1LPENR;     /*!< RCC AHB1 peripheral clock enable in low power mode register
  __IO uint32_t AHB2LPENR;     /*!< RCC AHB2 peripheral clock enable in low power mode register
  __IO uint32_t AHB3LPENR;     /*!< RCC AHB3 peripheral clock enable in low power mode register
  uint32_t      RESERVED4;     /*!< Reserved, 0x5C
  __IO uint32_t APB1LPENR;     /*!< RCC APB1 peripheral clock enable in low power mode register
  __IO uint32_t APB2LPENR;     /*!< RCC APB2 peripheral clock enable in low power mode register
  uint32_t      RESERVED5[2];  /*!< Reserved, 0x68-0x6C
  __IO uint32_t BDCR;          /*!< RCC Backup domain control register,
  __IO uint32_t CSR;           /*!< RCC clock control & status register,
  uint32_t      RESERVED6[2];  /*!< Reserved, 0x78-0x7C
  __IO uint32_t SSCGR;         /*!< RCC spread spectrum clock generation register,
  __IO uint32_t PLLI2SCFGR;    /*!< RCC PLLI2S configuration register,
  uint32_t      RESERVED7[1];  /*!< Reserved, 0x88
  __IO uint32_t DCKCFGR;       /*!< RCC Dedicated Clocks configuration register,
} RCC_TypeDef;
```

RCC Register

- **Reset and Clock Control (RCC)**
  - **Clock Control**
    RCC->CR (Control Register)
  - **Clock Configuration**
    RCC->CFGR (Configuration Register)
    RCC->PLLCFGR
  - **Peripheral Clock Enable**
    RCC->AHB1ENR



RCC : Internal Clock (HSI) for GPIO

Option 1. Internal Clock (HSI) based GPIO

1. Enable HSI and choose as SYSCLK source

- Enable **HSI:** (**RCC->CR: HSION=1**)
- Wait until **HSI** is stable and ready: (**RCC->CR: HSIRDY? 1**)
- Choose the system clock switch : (**RCC->CFGR: SW = 00)**
- Check if the selected source is correct: (**RCC->CFGR: SWS ? 00)**

```
// RCC Control Register (HSI)
RCC->CR |= ((uint32_t)RCC_CR_HSION);

// wait until HSI is ready
while ( (RCC->CR & (uint32_t) RCC_CR_HSIRDY) == 0 ) {;}

// Select HSI as system clock source
// RCC Configuration Register
RCC->CFGR &= (uint32_t)((uint32_t) ~(RCC_CFGR_SW));
RCC->CFGR |= (uint32_t)RCC_CFGR_SW_HSI;

// Wait till HSI is used as system clock source
while ((RCC->CFGR & (uint32_t)RCC_CFGR_SWS) != 0 ) {;} /

// RCC Peripheral Clock Enable Register
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
```

2. Configure APB/AHB Prescaler (optional)

- Change Prescaler: (**RCC->CFGR: HPRE, PPRE)**

3. Enable GPIOx clock(AHB1ENR)

- Enable (**RCC_AHB1ENR)** for PORTx

RCC : Internal Clock (HSI) for GPIO

<RM, Ch 6. Reset and clock control (RCC) >

Option 1. Internal Clock (HSI) based GPIO

1-1. Enable HSI

- Enable **HSI** (**RCC->CR: HSION=1**)
- Wait until **HSI** is stable and ready (**RCC->CR: HSIRDY? 1**)

6.3.1 RCC clock control register (RCC_CR)

Address offset: 0x00

Reset value: 0x0000 XX81 where X is undefined.

Access: no wait state, word, half-word and byte access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				PLLI2S RDY	PLLI2S ON	PLLRDY	PLLON	Reserved				CSS ON	HSE BYP	HSE RDY	HSE ON
				r	rw	r	rw					rw	rw	r	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
HSICAL[7:0]								HSITRIM[4:0]					Res.	HSI RDY	HSION
r	r	r	r	r	r	r	r	rw	rw	rw	rw	rw		r	rw

Bit 1 **HSIRDY**: Internal high-speed clock ready flag

0: HSI oscillator not ready
1: HSI oscillator ready

Bit 0 **HSION**: Internal high-speed clock enable

0: HSI oscillator OFF
1: HSI oscillator ON

```
#define RCC_CR_HSION     (0x1UL << 0U)
#define RCC_CR_HSIRDY    (0x1UL << 1U)
```

```
// Enable High Speed Internal Clock (HSI = 16 MHz)
RCC->CR |= ((uint32_t)RCC_CR_HSION);
// RCC->CR |= (0x1UL << 0U);
```

```
// Wait until HSI is ready
while ( (RCC->CR & (uint32_t) RCC_CR_HSIRDY) == 0 ) {;}
// while ( (RCC->CR & (0x1UL << 1U) ) == 0 ) {;}
```

RCC : Internal Clock (HSI) for GPIO

Option 1. Internal Clock (HSI) based GPIO

1-2. Choose HSI as SYSCLK source:

- Choose the system clock switch : **(RCC->CFGR : SW = 00)**
- Check if the selected source is correct: **(RCC->CFGR : SWS ? 00)**

6.3.3 RCC clock configuration register (RCC_CFGR)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MCO2		MCO2 PRE[2:0]			MCO1 PRE[2:0]			I2SSCR	MCO1		RTCPRE[4:0]				
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PPRE2[2:0]			PPRE1[2:0]			Reserved		HPRE[3:0]				SWS1	SWS0	SW1	SW0
rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	r	r	rw	rw

Bits 3:2 **SWS:** System clock switch status

Set and cleared by hardware to indicate which clock source is used as the system clock.

00: HSI oscillator used as the system clock

01: HSE oscillator used as the system clock

10: PLL used as the system clock

11: not applicable

Bits 1:0 **SW:** System clock switch

Set and cleared by software to select the system clock source.

Set by hardware to force the HSI selection when leaving the Stop or Standby mode or in case of failure of the HSE oscillator used directly or indirectly as the system clock.

00: HSI oscillator selected as system clock

01: HSE oscillator selected as system clock

10: PLL selected as system clock

11: not allowed

```
// Select HSI as system clock source
  RCC->CFGR &= (uint32_t)(~RCC_CFGR_SW);
  RCC->CFGR |= (uint32_t)RCC_CFGR_SW_HSI;

  // Wait till HSI is used as system clock source
  while ((RCC->CFGR & (uint32_t)RCC_CFGR_SWS) != 0 ) {;}
  //while ((RCC->CFGR & (0x3UL<< 2U) )!= 0 ) {;}
  //while (((RCC->CFGR >>2U) & (0x3UL) )!= 0 ) {;}
```

<stm32f411xe.h>

```
/*!< SW configuration */
#define RCC_CFGR_SW                          0x3UL << 0UL
#define RCC_CFGR_SW_HSI                      0x00000000U
#define RCC_CFGR_SW_HSE                      0x00000001U

/*!< SWS configuration */
#define RCC_CFGR_SWS                         (0x3UL<< 2U)
```

RCC : Internal Clock (HSI) for GPIO

Option 1. Internal Clock (HSI) based GPIO

3. (optional) Configure APB/AHB Prescaler

Change Prescaler: (**RCC->CFGR : HPRE, PPRE**)

Bits 7:4 **HPRE:** AHB prescaler

Set and cleared by software to control AHB clock division factor.

Caution: The clocks are divided with the new prescaler factor from 1 to 16 AHB cycles after HPRE write.

0xxx: system clock not divided
1000: system clock divided by 2
1001: system clock divided by 4
1010: system clock divided by 8
1011: system clock divided by 16
1100: system clock divided by 64
1101: system clock divided by 128
1110: system clock divided by 256
1111: system clock divided by 512

AHB clock = system clock = 84MHz

```
RCC->CFGR &= ~RCC_CFGR_HPRE;
RCC->CFGR &= ~RCC_CFGR_PPRE1;
RCC->CFGR |=  RCC_CFGR_PPRE1_2;
RCC->CFGR &= ~RCC_CFGR_PPRE2;
```

Bits 15:13 **PPRE2:** APB high-speed prescaler (APB2)

Set and cleared by software to control APB high-speed clock division factor.

Caution: The software has to set these bits correctly not to exceed 100 MHz on this domain. The clocks are divided with the new prescaler factor from 1 to 16 AHB cycles after PPRE2 write.

0xx: AHB clock not divided
100: AHB clock divided by 2
101: AHB clock divided by 4
110: AHB clock divided by 8
111: AHB clock divided by 16

APB2 clock = 84MHz/1 < 100MHz

Bits 12:10 **PPRE1:** APB Low speed prescaler (APB1)

Set and cleared by software to control APB low-speed clock division factor.

Caution: The software has to set these bits correctly not to exceed 50 MHz on this domain. The clocks are divided with the new prescaler factor from 1 to 16 AHB cycles after PPRE1 write.

0xx: AHB clock not divided
100: AHB clock divided by 2
101: AHB clock divided by 4
110: AHB clock divided by 8
111: AHB clock divided by 16

APB1 clock = 84MHz / 2 = 42MHz < 50MHz

RCC : Internal Clock (HSI) for GPIO

Option 1. Internal Clock (HSI) based GPIO

4. Enable GPIOx clock

- Enable (**RCC_AHB1ENR)** for PORTx

6.3.9 RCC AHB1 peripheral clock enable register (RCC_AHB1ENR)

Address offset: 0x30

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									DMA2EN	DMA1EN	Reserved				
									rw	rw					

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			CRCEN	Reserved				GPIOH EN	Reserved		GPIOEEN	GPIOD EN	GPIOC EN	GPIOB EN	GPIOA EN
			rw					rw			rw	rw	rw	rw	rw

```
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
```

(0x1UL << 0)

Bit 0 **GPIOAEN:** IO port A clock enable
Set and cleared by software.
0: IO port A clock disabled
1: IO port A clock enabled

```
#define RCC_AHB1ENR_GPIOAEN_Pos            (0U)
#define RCC_AHB1ENR_GPIOAEN_Msk            (0x1UL << RCC_AHB1ENR_GPIOAEN_Pos)   /*!< 0x00000001 */
#define RCC_AHB1ENR_GPIOAEN                RCC_AHB1ENR_GPIOAEN_Msk
```

RCC : PLL for GPIO

Option 2. PLL based GPIO Clock

1. Enable either (HSI, or HSE) for PLL and Choose PLL for System Clock

- Enable **HSI or HSE** — **RCC->CR : HSION=1 / HSEON=1**
- Wait until **HSI** or **HSE** is stable — **RCC->CR : HSIRDY? 1 / HSERDY? 1**
- Choose **PLL** for system clock switch : — **RCC->CFGR : SW = 10**
- Check if **PLL** selection is correct: — **RCC->CFGR : SWS ? 10**

2. Select the clock source for PLL

- Select PLL source(HSI or HSE): — **RCC->PLLCFGR : PLLSRC= 0 or 1**

3. (Optional)Configure PLL parameters

- Select (M/N/P): — **RCC->PLLCFGR : PLLM, PLLN, …**

4. Enable PLL

- Enable main PLL: — **RCC->CR : PLLON=1**

5. (Optional)Configure APB/AHB Prescaler

- Change Prescaler: — **RCC->CFGR : HPRE, PPRE**

6. Enable GPIOx clock(AHB1ENR)

- Enable (**RCC_AHB1ENR)** for PORTx

RCC : PLL for GPIO

Option 2. PLL based GPIO Clock

1. Enable either (HSI, or HSE) for PLL and Choose PLL for System Clock

- Enable **HSE or HSI** — **RCC->CR : HSION=1**
- Wait until **HSE** or **HSI** is stable — **RCC->CR : HSIRDY? 1**
- Choose PLL for system clock switch : — **RCC->CFGR : SW = 10**
- Check if PLL selection is correct: — **RCC->CFGR : SWS ? 10**

6.3.3 RCC clock configuration register (RCC_CFGR)

Address offset: 0x08

Reset value: 0x0000 0000

Access: 0 ≤ wait state ≤ 2, word, half-word and byte access

1 or 2 wait states inserted only if the access occurs during a clock source switch.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MCO2		MCO2 PRE[2:0]			MCO1 PRE[2:0]			I2SSCR	MCO1		RTCPRE[4:0]				
rw		rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PPRE2[2:0]			PPRE1[2:0]			Reserved		HPRE[3:0]				SWS1	SWS0	SW1	SW0
rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	r	r	rw	rw

Bits 3:2 **SWS:** System clock switch status

Set and cleared by hardware to indicate which clock source is used as the system clock.

00: HSI oscillator used as the system clock

01: HSE oscillator used as the system clock

10: PLL used as the system clock

11: not applicable

Bits 1:0 **SW:** System clock switch

Set and cleared by software to select the system clock source.

Set by hardware to force the HSI selection when leaving the Stop or Standby mode or in case of failure of the HSE oscillator used directly or indirectly as the system clock.

00: HSI oscillator selected as system clock

01: HSE oscillator selected as system clock

10: PLL selected as system clock

11: not allowed

RCC : PLL for GPIO

Option 2. PLL based GPIO Clock

2. Select the clock source for PLL

- Select the PLL source(HSI or HSE): **RCC->PLLCFGR : PLLSRC= 0 or 1**
- Need to disable PLL before selection

6.3.2 RCC PLL configuration register (RCC_PLLCFGR)

Address offset: 0x04

Reset value: 0x2400 3010

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				PLLQ3	PLLQ2	PLLQ1	PLLQ0	Reserved	PLLSRC	Reserved				PLLP1	PLLP0
				rw	rw	rw	rw		rw					rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	PLLN									PLLM5	PLLM4	PLLM3	PLLM2	PLLM1	PLLM0
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 22 **PLLSRC:** Main PLL(PLL) and audio PLL (PLLI2S) entry clock source

Set and cleared by software to select PLL and PLLI2S clock source. This bit can be written only when PLL and PLLI2S are disabled.

0: HSI clock selected as PLL and PLLI2S clock entry

1: HSE oscillator clock selected as PLL and PLLI2S clock entry

Example:

(PLL clock input source) = HSI clock = 16 MHz

```
RCC->PLLCFGR &= ~RCC_PLLCFGR_PLLSRC;
RCC->PLLCFGR |= RCC_PLLCFGR_PLLSRC_HSI;
```

RCC : PLL for GPIO

Option 2. PLL based GPIO Clock

3. (Optional)Configure PLL parameters

***p164 [RM]**

- Select (M/N/P): **RCC->PLLCFGR: PLLM, PLLN,** …
- Caution: VCO output frequency is between 100 and 432 MHz. (50 ≤ PLLN ≤ 432)
- Caution: VCO input frequency ranges from 1 to 2 MHz(recommend). (PLLM with 2 ≤ PLLM ≤ 63)
- Caution: PLL do not exceed 80MHz for this MCU (PLLP=2,4,6,8)

Bits 14:6 **PLLN:** Main PLL (PLL) multiplication factor for VCO

Bits 5:0 **PLLM:** Division factor for the main PLL (PLL) and audio PLL (PLLI2S) input clock

Bits 17:16 **PLLP:** Main PLL (PLL) division factor for main system clock

000110010: PLLN = 50 to 110110000: PLLN = 432

000010: PLLM = 2 to 111111: PLLM = 63

6.3.2 RCC PLL configuration register (RCC_PLLCFGR)

This register is used to configure the PLL clock outputs according to the formulas:

- $f_{(VCO\ clock)} = f_{(PLL\ clock\ input)} \times (PLLN / PLLM)$
- $f_{(PLL\ general\ clock\ output)} = f_{(VCO\ clock)} / PLLP$
- $f_{(USB\ OTG\ FS,\ SDIO)} = f_{(VCO\ clock)} / PLLQ$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				PLLQ3	PLLQ2	PLLQ1	PLLQ0	Reserved	PLLSRC	Reserved				PLLP1	PLLP0
				rw	rw	rw	rw		rw					rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	PLLN									PLLM5	PLLM4	PLLM3	PLLM2	PLLM1	PLLM0
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Example:
PLL clock source(HSI) = 16 MHz
PLLN=50
PLLM=8
PLLP=2

→ PLL clock=[(16/8)*50]/2=50MHz

RCC : PLL for GPIO

Option 2. PLL based GPIO Clock

3. (Optional)Configure PLL parameters

Bits 5:0 **PLLM:** Division factor for the main PLL (PLL) input clock

Set and cleared by software to divide the PLL input clock before the VCO. These bits can be written only when the PLL is disabled.

Caution: The software has to set these bits correctly to ensure that the VCO input frequency ranges from 1 to 2 MHz. It is recommended to select a frequency of 2 MHz to limit PLL jitter.

VCO input frequency = PLL input clock frequency / PLLM with 2 ≤PLLM ≤63

000000: PLLM = 0, wrong configuration
000001: PLLM = 1, wrong configuration
000010: PLLM = 2
000011: PLLM = 3
000100: PLLM = 4

PLLM = 8

Bits 14:6 **PLLN:** Main PLL (PLL) multiplication factor for VCO

Set and cleared by software to control the multiplication factor of the VCO. These bits can be written only when PLL is disabled. Only half-word and word accesses are allowed to write these bits.

Caution: The software has to set these bits correctly to ensure that the VCO output frequency is between 100 and 432 MHz. (check also *Section 6.3.20: RCC PLLI2S configuration register (RCC_PLLI2SCFGR)*)

VCO output frequency = VCO input frequency × PLLN with 50 ≤PLLN ≤432

000000000: PLLN = 0, wrong configuration
000000001: PLLN = 1, wrong configuration
...
000110010: PLLN = 50
...
001100011: PLLN = 99
001100100: PLLN = 100
...

PLLN = 84

Bits 17:16 **PLLP:** Main PLL (PLL) division factor for main system clock

Set and cleared by software to control the frequency of the general PLL output clock. These bits can be written only if PLL is disabled.

Caution: The software has to set these bits correctly not to exceed 100 MHz on this domain.

PLL output clock frequency = VCO frequency / PLLP with PLLP = 2, 4, 6, or 8

00: PLLP = 2
01: PLLP = 4
10: PLLP = 6
11: PLLP = 8

- f(VCO clock) = f(PLL clock input) x (PLLN/PLLM)
  = 16MHz x (84 / 8)
  = 168MHz
- f(PLL general clock output) = f(VCO clock) / PLLP
  = 168MHz / 2
  = **84MHz**

```
RCC->PLLCFGR = (RCC->PLLCFGR & ~RCC_PLLCFGR_PLLN) | 84U << 6;
RCC->PLLCFGR = (RCC->PLLCFGR & ~RCC_PLLCFGR_PLLM) | 8U ;
RCC->PLLCFGR &= ~RCC_PLLCFGR_PLLP;
```

RCC : PLL for GPIO

Option 2. PLL based GPIO Clock

4. Enable PLL

- Enable main PLL: **RCC->CR: PLLON=1**

Bit 25 **PLLRDY**: Main PLL (PLL) clock ready flag
Set by hardware to indicate that PLL is locked.
0: PLL unlocked
1: PLL locked

Bit 24 **PLLON**: Main PLL (PLL) enable
Set and cleared by software to enable PLL.
Cleared by hardware when entering Stop or Standby mode. This bit cannot be reset if PL
clock is used as the system clock.
0: PLL OFF
1: PLL ON

6.3.1 RCC clock control register (RCC_CR)

Address offset: 0x00

Reset value: 0x0000 XX81 where X is undefined.

Access: no wait state, word, half-word and byte access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				PLLI2S RDY	PLLI2S ON	PLLRDY	PLLON	Reserved				CSS ON	HSE BYP	HSE RDY	HSE ON
				r	rw	r	rw					rw	rw	r	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
HSICAL[7:0]								HSITRIM[4:0]					Res.	HSI RDY	HSION
r	r	r	r	r	r	r	r	rw	rw	rw	rw	rw		r	rw